



blackhat.pcap

No.	Time	Source	Destination	Protocol	Length	Info
700	1.959762	127.0.0.1	127.10.20.30	HTTP	372	PUT /nikto-test-tUwLdiu5.html HTTP/1.1 (application/x-www-form-urlencoded)
1110	121.114685	127.0.0.1	127.10.20.30	HTTP	1755	PUT / RkYgRDggRkYgRTAgMDAgMTAgNEEgNDYgNDkgNDYgMDAgMDEgMDEgMDEgMDAgNkEgMDAgNkEgMDAg
1854	271.877817	127.0.0.1	127.10.20.30	HTTP	372	PUT /nikto-test-SXAINCH.html HTTP/1.1 (application/x-www-form-urlencoded)

Apresenta um código:

RkYgRDggRkYgRTAgMDAgMTAgNEEgNDYgNDkgNDYgMDAgMDEgMDEgMDEgMDAgNkEgMDAgNkEgMDAg

MDAgRkYgREIgMDAgNDMgMDAgMDMgMDIgMDIgMDMgMDIgMDIgMDMgMDMgMDMgMDMgMDQgMDMgMDMg

MDQgMDUgMDggMDUgMDUgMDQgMDQgMDUgMEEgMDcgMDcgMDYgMDggMEMgMEEgMEMgMEMgMEIgMEEg

MEIgMEIgMEQgMEUgMTIgMTAgMEQgMEUgMTEgMEUgMEIgMEIgMTAgMTYgMTAgMTEgMTMgMTQgMTUg

MTUgMTUgMEMgMEYgMTcgMTggMTYgMTQgMTggMTIgMTQgMTUgMTQgRkYgREIgMDAgNDMgMDEgMDMg

Onde nos gerou um JPEG aparentemente.

[illegible]

Utilizado o xxd para gerar essa imagem:

```
(root@DESKTOP-EM0E755)-[/mnt/f/Downloads]
# xxd -r -p hex.txt > arquivo_secreto.jpg
```

Mas está quebrado a imagem.

Utilizando o filtro do wireshark por arquivos mais pesados.

Wireshark · Export · HTTP object list

Text Filter: Content Type: All Content-Types

Packet	Hostname	Content Type	Size	Filename
1709	127.10.20.30	text/html	10 kB	/
1711	127.10.20.30	text/html	10 kB	/
1713	127.10.20.30	text/html	10 kB	/
1743	127.10.20.30	text/html	10 kB	index.html
1785		text/html	10 kB	/
1830	127.10.20.30	text/html	10 kB	/
2797	127.10.20.30	text/html	10 kB	/
2801	127.10.20.30	text/html	10 kB	/
2807	127.10.20.30	text/html	10 kB	/
2809	127.10.20.30	text/html	10 kB	/
2937	127.10.20.30	text/html	10 kB	/
3037	127.10.20.30	text/html	10 kB	/
1110			1,605 bytes	/
569	127.10.20.30	text/html	469 bytes	/
1722	127.10.20.30	text/html	469 bytes	/
2939	127.10.20.30	text/html	304 bytes	shadow
125	127.10.20.30	text/html	301 bytes	2IBwrxU3.php#
667		text/html	301 bytes	.
679		text/html	301 bytes	/
1112		text/html	301 bytes	/
1293	127.10.20.30	text/html	301 bytes	9qHwudjN.php#
1821		text/html	301 bytes	.
1833		text/html	301 bytes	/
2972		text/html	301 bytes	/
2985	127.10.20.30	text/html	300 bytes	/
701	127.10.20.30	text/html	298 bytes	nikto-test-tUwLd1u5.html
1855	127.10.20.30	text/html	298 bytes	nikto-test-SXAINkCM.html
2982		text/html	297 bytes	/
2953		text/html	285 bytes	

Buttons: Save, Save All, Preview, Close, Help

Onde podemos pegar exatamente a comunicação feita:

```
Host: 127.10.20.30
sec-ch-ua: "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.5672.93 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Connection: close
```

[illegible]

Voltado e filtrado por conexões SYN, ACK e colocado no filtro expert para analisarmos, vimos uma porta estranha sendo usada:

Display filter: "tcp.flags.syn == 1"
☒ Limit to Display Filter ☒ Group by summary Search:

[Packet list]		[Display filter]		tcp.connection.synack			
Options:		[Narrow & Wide]		☐ Case sensitive		☐ Backwards	
						☐ Multiple occurrences	
No.	Time	Source	Destination	Protocol	Length	Info	
1138	229.231975	127.0.0.30	127.0.0.1	TCP	66	11111 → 41882 [FIN, ACK] Seq=73983 Ack=2 Win=65536 Len=0 TSval=2393523355 TSecr=415688164	
1139	229.236875	127.0.0.30	127.0.0.0-1	TCP	66	11111 → 41882 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=8192 TSval=2393529966 TSecr=415687578	
1135	229.232396	127.0.0.30	127.0.0.1	TCP	8284	11111 → 41882 [PSH, ACK] Seq=40961 Ack=1 Win=65536 Len=32768 TSval=2393529967 TSecr=415687578	
1136	229.232418	127.0.0.30	127.0.0.1	TCP	320	11111 → 41882 [PSH, ACK] Seq=73729 Ack=1 Win=65536 Len=254 TSval=2393529967 TSecr=415687578	
1137	229.235444	127.0.0.30	127.0.0.1	TCP	32634	11111 → 41882 [PSH, ACK] Seq=61339 Ack=1 Win=65536 Len=25408 TSval=2393529969 TSecr=415687578	
1128	229.229698	127.0.0.30	127.0.0.1	TCP	74	11111 → 41882 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 TSval=2393529965 TSecr=415687578 WS=128	
1129	229.229755	127.0.0.1	127.0.0.30	TCP	66	41882 → 11111 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4156875776 TSecr=2393529965	
1133	229.231635	127.0.0.1	127.0.0.30	TCP	66	41882 → 11111 [ACK] Seq=1 Ack=40961 Win=24576 Len=0 TSval=4156875778 TSecr=2393529966	
1137	229.235444	127.0.0.30	127.0.0.1	TCP	66	41882 → 11111 [ACK] Seq=1 Ack=73983 Win=41872 Len=0 TSval=4156875779 TSecr=2393529967	
1133	229.231566	127.0.0.1	127.0.0.30	TCP	66	41882 → 11111 [ACK] Seq=1 Ack=8193 Win=57344 Len=0 TSval=4156875778 TSecr=2393529966	
1124	229.620869	127.0.0.1	127.0.0.30	TCP	66	41882 → 11111 [ACK] Seq=2 Ack=73984 Win=65536 Len=0 TSval=4156888166 TSecr=2393533355	
1134	229.618146	127.0.0.1	127.0.0.30	TCP	66	41882 → 11111 [FIN, ACK] Seq=1 Ack=73983 Win=65536 Len=0 TSval=4156888164 TSecr=2393529967	
1127	229.618113	127.0.0.30	127.0.0.1	TCP	74	41882 → 11111 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=65496 SACK_PERM_TSval=4156875776 TSecr=WS=128	
1134	229.232315	127.0.0.1	127.0.0.30	TCP	66	[TCP Window Update] 41882 → 11111 [ACK] Seq=1 Ack=40961 Win=65536 Len=0 TSval=4156875778 TSecr=2393529966	

[illegible]

Convertido para hex dump e jogado para um arquivo .txt

FF DA	início dos dados da imagem — SOS
-------	----------------------------------

FF D9	fim do JPEG — EOI
-------	-------------------

O problema é que **não existe** FF D8 **no começo**.

Isso indica que o arquivo está **truncado/corrompido no início**, mas o restante contém uma imagem JPEG.

Onde identificamos o trecho:

```
00000160  fa ff da 00 0c 03 01 00 02 11 03 11 00 3f 00 f8
```

Depois dele começa a informação comprimida da imagem.

Um JPEG normal teria algo parecido com:

```
FF D8
FF E0 ...
JFIF
...
FF DB ...
FF C0 ...
FF C4 ...
FF DA
```

No nosso dump, começamos praticamente no meio dessas estruturas.

A estrutura recuperada precisava receber um cabeçalho antes dos bytes fornecidos.

Onde feito a conversão do hexdump:

```
awk '{
    for (i=2; i<=17 && $i ~ /^[0-9A-Fa-f][0-9A-Fa-f]$/; i++)
        printf "%s", $i
    printf "\n"
}' dump.txt > payload.hex
```

E depois para binário:

```
xxd -r -p payload.hex payload.bin
```

e criado nosso header:

GNU nano 8.7.1 header.hex

```
ffd8ffe000104a46494600010100000100010000ffdb004300080606070605080707070909080a0c140d0c0b0b0c1912130f141d1a1f1e1d1a1c1c2>
```

Read 1 line

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy

[illegible]

E convertido para bin.

```
xxd -r -p header.hex header.bin
```

Onde apresenta ser o jpg.

```
(root@ DESKTOP-EM0E755)-[/mnt/f/Downloads/New Folder]
# ls -lh header.bin
-rwxrwxrwx 1 root root 259 Sep 27 22:44 header.bin

(root@ DESKTOP-EM0E755)-[/mnt/f/Downloads/New Folder]
# xxd -l 32 header.bin
00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001  ....JFIF.....
00000010: 0001 0000 ffd9 0043 0008 0606 0706 0508  ....C.....
```

O cabeçalho reconstruído ficou:

```
FF D8
FF E0 00 10 4A 46 49 46 ...
```

```
FF D8
FF E0 00 10 4A 46 49 46 ...
```

```
FF DB ...  
FF DB ...  
FF C0 ...  
FF C4 ...  
FF C4 ...
```

O ponto importante é que **não alteramos os dados da imagem que estavam no desafio**.
Apenas reconstruímos a parte inicial que estava ausente.

No cabeçalho reconstruído aparece:

```
FF C0 00 11 08 01 18 01 A0 ...
```

Onde interpretado:

```
01 18 = 280  
01 A0 = 416
```

Sendo altura e largura.

No JPEG recuperado, o deslocamento fica:

```
00000100 42 b1 c1 15 52 d1 f0 24 ...
```

O cabeçalho adicionado tinha **0x100 bytes = 256 bytes**.

Agora juntando os dois:

```
cat header.bin payload.bin > recuperado.jpg
```

Depois de juntar:

```
HEADER_JPEG + DUMP
```

o arquivo passa a possuir:

```
FF D8    → início  
...  
FF DA    → dados da imagem  
...  
FF D9    → final
```

Então ele pode ser aberto normalmente como JPEG.



Resposta: FIAP{WIR3SH4RK}